

Customer Churn Survival Model

A survival-analysis module for a foodservice distributor that predicts when a customer will stop ordering, produces a per-customer time-to-churn curve, and explains the risk with the specific factors driving it.

Andrew McCombs · Independent Portfolio Project · Phoenix, AZ

Status. Independent portfolio project, not connected to any employer or commercial product. Built on anonymized distributor order data to demonstrate survival-analysis system design, label-and-leakage discipline, and explainability for adoption by sales teams. Full pipeline and interface exist. The model deliberately blocks training and deployment until the data clears a minimum event count and concordance floor, so no accuracy is reported today.

1. The problem

A foodservice distributor does not lose a customer in a single dramatic moment. The customer orders a little less often, drops a few product lines, shifts spend to a competitor, and one quarter simply does not come back. By the time a sales rep notices, the account is gone.

The useful question is therefore not “who churned” but “who is about to, how soon, and why,” with enough lead time to act.

Churn modeling in this domain has to clear several bars that a simple flag-inactive-accounts rule does not.

Most customers are censored

At any moment the large majority of customers have not churned. They are still active, so their final tenure is unknown. Throwing them away wastes most of the data, and treating “has not churned yet” as “will not churn” is wrong. Survival analysis is the correct frame precisely because it handles right-censored observations: an active customer contributes the information that they survived at least this long, without pretending to know the rest.

Churn is a date, not a flag

A distributor schedules a recovery call differently for an account at 70 percent risk in two weeks than one at 70 percent risk in six months. A classifier that outputs a single probability throws away the timing that makes the prediction actionable. A survival curve keeps it.

The label is genuinely ambiguous

A school that stops ordering in June has not churned, it has gone on summer break. A restaurant that pauses for a remodel has not churned. Defining churn naively produces a flood of false positives every summer and destroys trust in the tool. The definition has to encode the seasonality of the business.

Leakage is easy and fatal

The features that best predict churn (orders falling off, spend collapsing) are the same signals that appear because the customer is already leaving. If the model trains on features measured after the churn event, it will look brilliant in validation and useless in production. Preventing that is a data-engineering discipline, not a modeling choice.

2. The honest finding

The hard part of churn is not the model. The model class itself is a well-understood tool.

Deciding what counts as churn in a business with seasonal closures and customers who leave and come back, and then making sure the model never trains on information from after the event it is trying to predict, is where the real engineering went.

The system is deliberately built to refuse to train, or to deploy, until it has the labeled events and the concordance to justify it. A model that says “not enough events to train yet” is more valuable to an operator than one that reports a confident concordance built on twelve churn events.

3. System architecture

The module is five pipelines and a dashboard surface, layered onto an existing reorder system so it reuses the same order data and customer configuration.

Pipelines

- ▶ **1. Label construction (weekly).** Walks each customer's order history and assigns a churn label and a tenure, with right-censoring for active customers.
- ▶ **2. Feature pipeline (weekly).** Computes one row per active customer across order, product-mix, private-label, service, and relationship signals, and appends a timestamped snapshot to a history table.
- ▶ **3. Model training (monthly).** Joins labels to the correct historical feature snapshot, fits the survival model, evaluates concordance, and registers the artifact only if it clears the quality floor.
- ▶ **4. Scoring (weekly).** Loads the active model, scores every customer, and writes the survival curve plus the top risk factors.
- ▶ **5. Runtime feedback loop.** Lets the reorder engine lower its confidence threshold for high-risk customers so an at-risk account sees its suggested cart more readily, without ever mutating stored settings.

Training is monthly and scoring is weekly on purpose. Churn patterns change slowly, so retraining weekly on a small number of events produces unstable models. Scoring is cheap and worth running often.

Technology stack

Layer	Choice
Language	Python 3.12
Survival model	scikit-survival (GradientBoostingSurvivalAnalysis)
Explainability	SHAP (permutation explainer)
Store	Postgres (Supabase) with row-level security
Interface	Next.js admin dashboard, async server components
Recommendation layer	Claude (Sonnet) for per-customer intervention text
Orchestration	Scheduled jobs alongside the existing reorder pipelines

4. Defining churn, and not leaking

The churn definition

Churn is defined year over year against completed calendar months. A customer who ordered in a given month last year but placed no order in the same month this year is treated as churned, dated to that month. Only months that have fully ended are ever evaluated, so the current in-progress month is never mistaken for a loss.

This definition is robust to the seasonality that breaks simpler rules: a summer-only account is compared against its own prior summer, not against a winter baseline. Customers with fewer than three lifetime orders are skipped as too thin to model, and customers with under twelve months of history are right-censored as active rather than labeled, because no prior year exists to compare against.

Re-acquisition

A customer who churns and then comes back within a twelve-month window is detected explicitly. The original churn label is marked superseded and a new tenure row is opened from the re-acquisition date, so the model learns from both the loss and the win as independent observations rather than silently overwriting one with the other.

Temporal leakage prevention

This is the part that earns its keep. For every churn event, the label builder records a feature snapshot date one week before the churn date. At training time the model does not join to today's features, it joins to the historical feature snapshot at or before that date, pulled from an accumulating history table by a lateral join. Active, censored customers use their most recent snapshot.

The result is that training features are always from before the event the model is asked to predict. The train/validation split is then time-based, not random: the most recent fifth of customers by snapshot date are held out, so the model is always validated on a period it could not have seen.

5. The model

The model is GradientBoostingSurvivalAnalysis from scikit-survival, gradient boosting fit to a Cox proportional-hazards loss. It was chosen over plain Cox regression because it captures interactions between features without hand-specification, and over a classifier because it outputs a full survival function and handles censoring correctly. Labels are passed as a structured survival array of (event occurred, duration), so censored customers contribute their partial tenure honestly.

The training run is conservative by design for a small-data setting: shallow trees, a low learning rate, subsampling, and a minimum-leaf size that scales with the number of churn events so the model cannot grow leaves finer than the data supports.

Quality gates

Performance is measured with the concordance index, the survival-analysis analogue of AUC, which asks how often the model ranks a customer who churned sooner as higher risk than one who churned later. The pipeline enforces quality gates around it.

Concordance	Behavior
0.70 or higher	Deploy
0.60 to 0.70	Warn but allow
Below 0.60	Abort, keep previous model
Fewer than 50 events	Refuse to train

The system refuses to train at all on fewer than fifty observed churn events, because a survival model fit on a handful of events reports a concordance that means nothing.

6. Features and signals

Features are grouped so that missing data in one group degrades gracefully instead of dropping a customer.

- ▶ **Order trajectory.** Frequency and spend over 30, 90, and 180 days, short-to-long ratios, days since last order, missed scheduled order days.
- ▶ **Product mix.** SKU and category diversity and their trends.
- ▶ **Private-label penetration.** House-brand share of spend and how deep it goes. A customer weaning off the distributor's own brands is leaving.
- ▶ **Reorder-system engagement.** Suggested-cart acceptance rate, consecutive dismissals.
- ▶ **Customer-service metrics.** Fill rate, credit rate, short-ship and dispute counts, days-to-pay trend.
- ▶ **Relationship signals.** Days since the last business review, sales visits, rep tenure, whether the rep changed in the last 90 days.

Lines, not cases

One signal proved more useful than the obvious one. Cases per delivery are seasonal and noisy, but lines per delivery (the number of distinct products on a drop) are stable year round. A customer quietly buying fewer product lines per delivery is consolidating their purchasing elsewhere. Lines are tracked as a first-class feature for that reason.

Imputation discipline

Because Cox models cannot ingest nulls, imputation is mandatory and is defined per feature group, with sentinel values that carry meaning (no business review on record becomes a large “no relationship activity” number, not a zero). Crucially, the exact same imputation code is shared between training and scoring. If train-time and score-time imputation diverged, the predictions would be quietly invalid, so the logic lives in one helper that both call.

7. Explainability

A risk score that a sales rep cannot interrogate will not be trusted or used.

The survival model has no native SHAP interface, so the predict function is wrapped to return a scalar risk (one minus the probability of surviving past the eight-week horizon) and a permutation SHAP explainer runs against that wrapper. At scoring time the same explainer runs per customer, and the top five contributing factors are stored alongside the curve.

The dashboard turns those into plain-language chips (a service issue, a rep change, an order-pattern decline) and a short recommended action, so the output a rep sees is “this account is at risk because fill rate slipped and there has been no business review in four months,” not an unexplained percentage.

8. Privacy and safety

Churn scores are admin-only and must never be visible to a customer. Every new table carries row-level security restricting it to the distributor's admins, and that constraint is treated as non-negotiable.

The reorder feedback loop is built to be safe by construction too: when a customer's risk is high the trigger engine lowers its effective confidence threshold at runtime only. It never writes that change back to the stored customer settings and it logs the override and the risk that caused it, so the behavior is auditable and reversible rather than a silent mutation of configuration.

9. Limitations and next steps

The honest status of this project is that the engineering is complete and the model is not yet validated. The full pipeline (labels, features with history, training, scoring), the database schema, and the admin interface (survival-curve chart, per-customer risk card, an at-risk-accounts panel) all exist.

What does not exist yet is a trained model artifact with a real concordance, because that requires at least fifty observed churn events and ideally eighteen months or more of order history loaded. The

system is deliberately built to block training and deployment until that bar is met, which is the correct behavior but means there is no accuracy number to report today.

Next steps

- ▶ 1. Load the full historical order set and run label construction to confirm the churn-event count clears the floor.
- ▶ 2. Train and read the first honest concordance.
- ▶ 3. The customer-service and engagement feature groups are wired but inert until that data is imported. The model is designed to pick them up automatically on the next monthly retrain once it lands.

10. Closing thought

The interesting result here is not a model score, because there is not one yet. It is that a credible churn system in this domain is mostly a labeling and leakage problem, and the right engineering response is to spend the effort on a churn definition that survives seasonality, on a re-acquisition rule, and on a snapshot discipline that guarantees the model never sees the future.

A model that says “not enough events to train yet” is more valuable to an operator than one that reports a confident concordance built on twelve churn events.